Jala University

DAS — Propuesta arquitectónica

Oscar Steven Muñoz Fajardo

Universidad Panamericana de Ciencias Aplicadas, UPCA

Arquitectura de Software 3 · JALA University · 2026

Agenda

1. El problema
2. ¿Qué necesita DAS?
3. Drivers arquitectónicos
4. La decisión: Monolito Modular
5. Bounded contexts
6. Fase 1 → Fase 2
7. Flujos principales
8. NIS y procesos de larga duración
9. Síncrono vs. asíncrono
10. Resiliencia y fallos
11. Escenario: NIS falla
12. Dispositivos offline
13. Datos y consistencia
14. Observabilidad
15. Riesgos y mitigaciones
16. Trade-offs
17. Evolución
18. Roadmap
19. Reflexión crítica — IA
20. Cierre

El problema

La Universidad Panamericana de Ciencias Aplicadas (UPCA) opera en:

10 ciudades
20 campus

5.000 estudiantes
400 profesores

+15.000 usuarios
en 2 años

El sistema de asistencia actual no puede verificar identidad de forma confiable, no opera sin conexión y no tiene un registro auditable a largo plazo. DAS debe modernizar ese ciclo completo: **autorregistro → verificación → credencial → asistencia → reportes**.

¿Qué necesita DAS?

Capacidades funcionales

- Autorregistro con documento y fotografía
- Verificación de identidad vía NIS
- Emisión de credencial DAS
- Captura de asistencia (entrada/salida)
- Soporte offline en dispositivos de campus
- Portal de consulta, reportes y estadísticas
- Notificaciones al usuario

Restricciones críticas

- NIS puede tardar **hasta 2 horas**
- Dispositivos pueden **perder conectividad**
- Reportes no deben bloquear la captura
- Retención mínima de **5 años** de asistencia
- Seguridad y confidencialidad de datos sensibles
- Equipo operativo de **3 a 5 desarrolladores**

Drivers arquitectónicos

Los drivers son condiciones que **cambian la estructura** del sistema, no solo una lista de características.

Driver	Exigencia del problema	Consecuencia arquitectónica
NIS asíncrono	Verificación tarda hasta 2 h	Proceso persistido con estados y recuperación
Asistencia near real-time	Entrada/salida debe ser inmediata	Captura: ruta liviana, no espera NIS
Escalabilidad	10.000 usuarios nuevos en 2 años	Separar lectura/escritura, extracción selectiva
Confiabilidad offline	Dispositivos pierden conectividad	Buffer local, idempotencia, reconexión controlada
Retención legal	5 años mínimo de asistencia	Políticas por capas: caliente / tibia / fría

La decisión arquitectónica

Fase 1 — Monolito Modular

- Un único despliegue
- Límites de dominio claros internamente
- Operación manejable para 3-5 devs
- Evolución gradual basada en evidencia

¿Por qué no microservicios desde el inicio?

Broker, red interna, observabilidad distribuida y varias piezas operables **antes de que la carga lo justifique**. El costo operativo no tiene un beneficio comprobado para el escenario actual.

Comparación

Monolito modular

✓ Un despliegue

✓ Límites claros

✓ Operación simple

✕ Escala como unidad

Microservicios

✓ Escala independiente

✕ Complejidad operativa

✕ Justificada por carga, no por diseño

¿Cómo está organizado DAS?

Cinco bounded contexts con responsabilidades, reglas y ownership de datos propios. En Fase 1 conviven en un mismo proceso con esquemas lógicos separados.

Registro y Verificación

Autorregistro, NIS, estados, aprobación admin

Identidad y Credenciales

DAS ID, insignia, sesión, revocación

Captura de Asistencia

Escaneos, offline, idempotencia

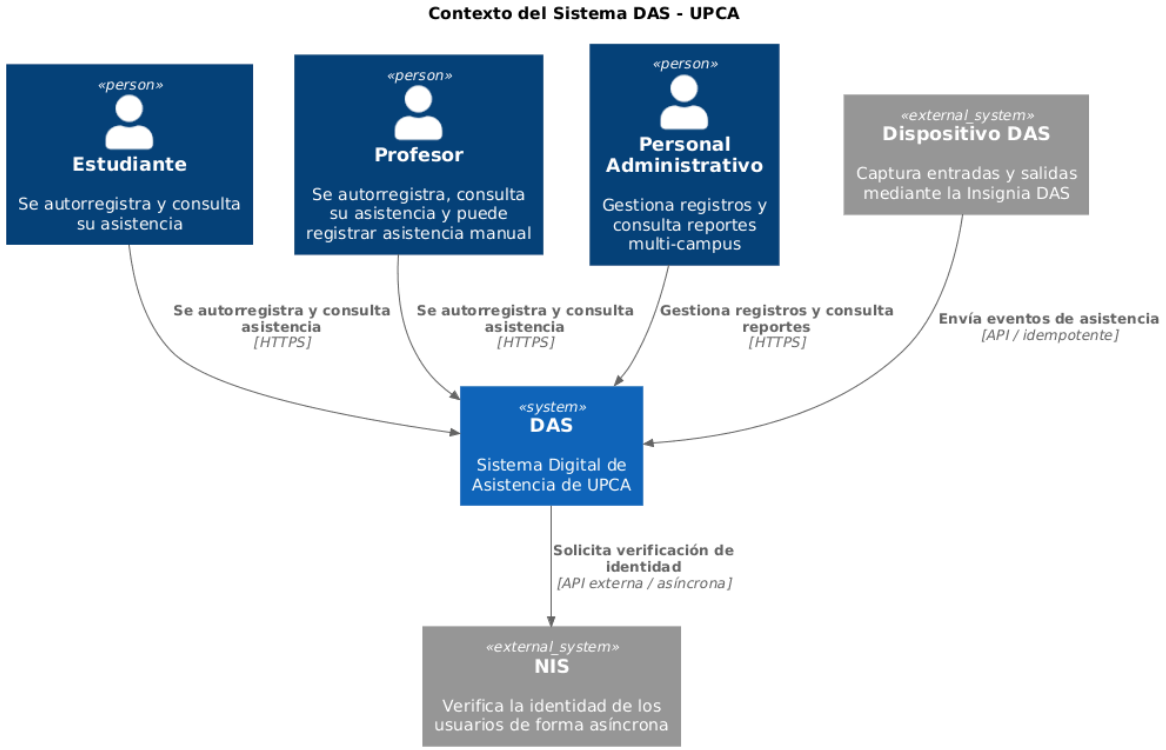
Reportes y Analítica

Consultas, estadísticas, proyecciones

Notificaciones

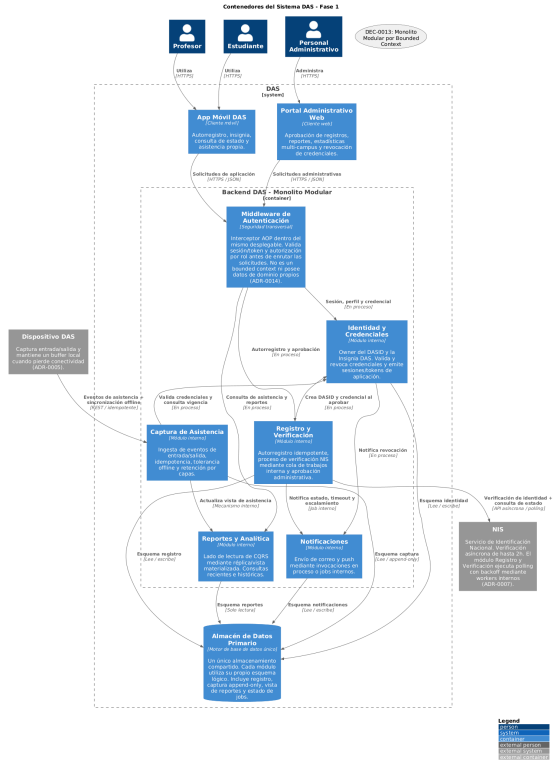
Email, push, estado de entrega

Diagrama de contexto (C4)



Ver diagrama completo

Diagrama de contenedores (C4)



Ver diagrama completo

Evolución: Fase 1 → Fase 2

Fase 1 · HOY

- Monolito modular
- Un proceso, un despliegue
- Motor de datos compartido
- Ownership **lógico** por módulo
- Escalado de toda la aplicación

→

Fase 2 · SI HAY EVIDENCIA

- Monolito + servicio(s) extraídos
- Primer candidato: **Captura de Asistencia**
- APIs o eventos entre fronteras reales
- Database per Service del módulo extraído
- Broker solo si pub/sub lo requiere

No es una migración Big Bang. No todos los módulos se convierten en servicios. La extracción depende de métricas reales.

NIS y procesos de larga duración

El flujo de registro

Usuario envía datos
↓
DAS responde **202 Accepted** + registrold
↓
Solicitud queda en `pendiente_nis`
↓
Job persistido de verificación
↓
ACL ↔ NIS externo (hasta 2 h)
↓
verificado / rechazado / escalado_manual
↓
aprobación administrativa
↓
DAS ID y credencial emitida

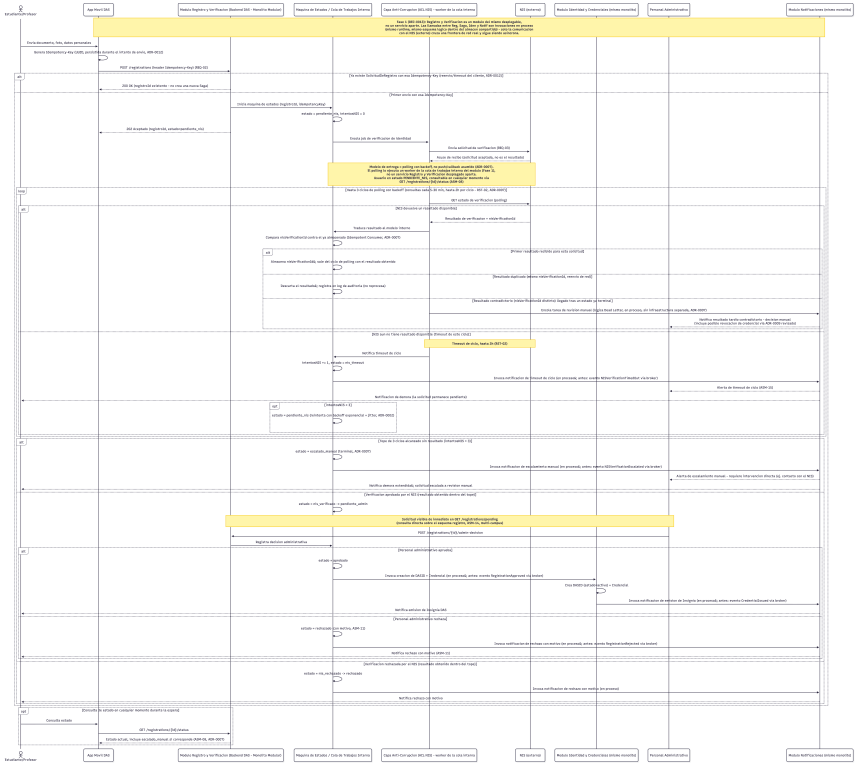
¿Por qué es importante?

DAS **no mantiene una petición HTTP abierta** durante la verificación. El usuario puede:

- Consultar el estado en cualquier momento
- Recibir notificación al completarse
- El backend puede reiniciarse sin perder el proceso
- Si NIS no responde → escalamiento manual, no bloqueo

La solicitud tiene estados explícitos:
`pendiente_nis` → `pendiente_admin` → `aprobado` / `rechazado` / `escalado_manual`

Diagrama de secuencia: Registro DAS



Ver diagrama completo

Síncrono vs. asíncrono

Síncrono

El cliente necesita respuesta inmediata

- App / portal → DAS
- Dispositivo → Captura (escaneo online)
- Validación y confirmación de persistencia

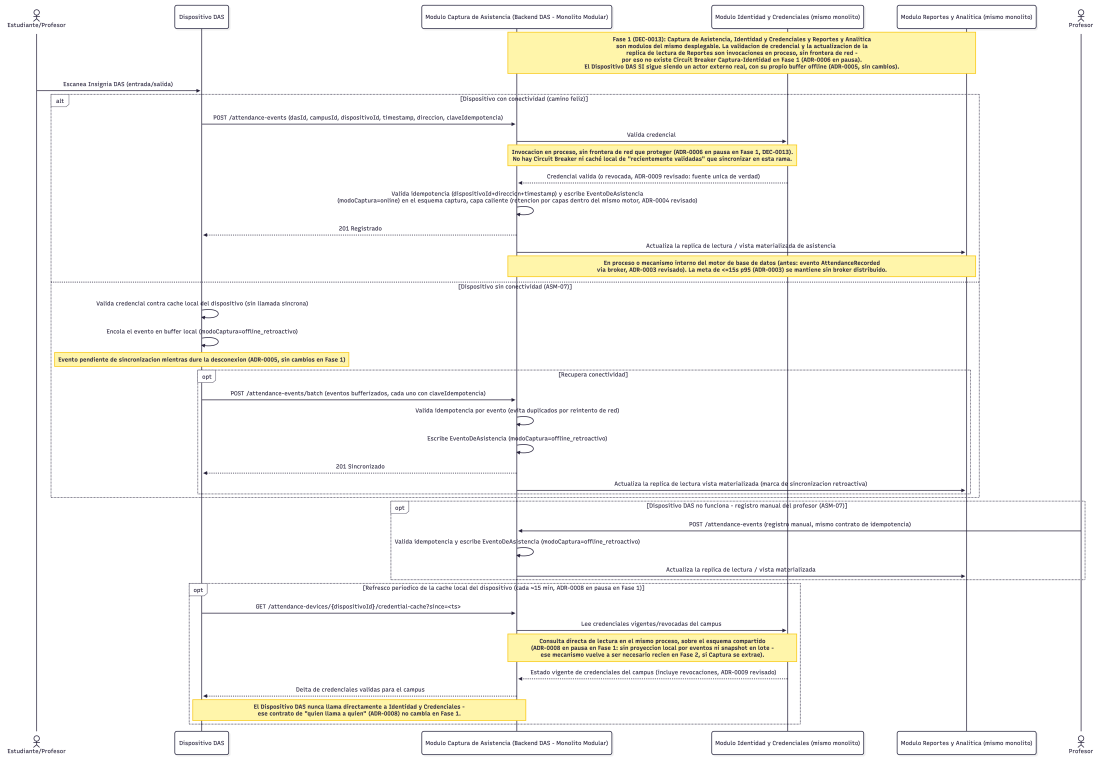
Asíncrono

El resultado tarda o no debe bloquear

- Registro → NIS (hasta 2 horas)
- Captura → Reportes (proyección)
- Dispositivo offline → Captura (batch)
- Email / push → Notificaciones

La regla: **síncrono donde la respuesta importa en el momento; asíncrono donde el resultado puede esperar sin bloquear la operación crítica.**

Flujo de captura de asistencia



Ver diagrama completo

Resiliencia: ¿qué pasa cuando algo falla?

Fallo	Comportamiento	Protección
NIS lento o no disponible	Registro queda pendiente	Job persistido · Retry · Escalamiento manual
DAS o worker se reinicia	Proceso en curso se interrumpe	Estado persistido; otro ciclo lo retoma
Dispositivo offline	Escaneos no llegan al backend	Buffer local · Caché · Lote · Idempotencia
Reconexión masiva	Lotes compiten con escaneos en vivo	Bulkhead · Throttling · Jitter · Prioridad captura
Proyección lenta	Reportes con retraso	Tarea durable · Lag medido · Reproyección
Email / push falla	Usuario no recibe notificación	Retry · DLQ (tabla persistida de fallos)

Reintentos solo para fallos **transitorios**. Datos inválidos o rechazos definitivos **no** se reintentan. Toda operación repetible usa idempotencia y backoff exponencial con jitter.

Escenario: NIS no responde

DAS intenta contactar NIS
↓
Timeout (sin respuesta)
↓
Retry + Backoff exponencial + jitter
↓
Reintentos agotados
↓
Circuit Breaker (falla rápido)
↓
Solicitud → `escalado_manual`
↓
Admin notificado · Proceso protegido
↓
Cuando NIS vuelve → retoma desde estado guardado

Garantías clave

- El proceso **no desaparece**; queda en estado auditado
- La asistencia sigue funcionando normalmente
- El admin puede intervenir manualmente
- Cuando NIS recupera, el job se retoma
- El Circuit Breaker protege al sistema de fallos en cascada

Ambigüedad reconocida

El contrato exacto de entrega de resultado de NIS (webhook, polling, correo) **no está definido** por el enunciado. La arquitectura soporta resultado diferido sin asumir un mecanismo específico.

Dispositivos offline y reconexión

Flujo offline

Dispositivo pierde conectividad
↓
Valida contra caché local
↓
Eventos acumulados en buffer local
↓
Reconexión
↓
Sincronización por lotes (batch)
↓
Idempotencia evita duplicados
↓
Eventos auditados con doble timestamp

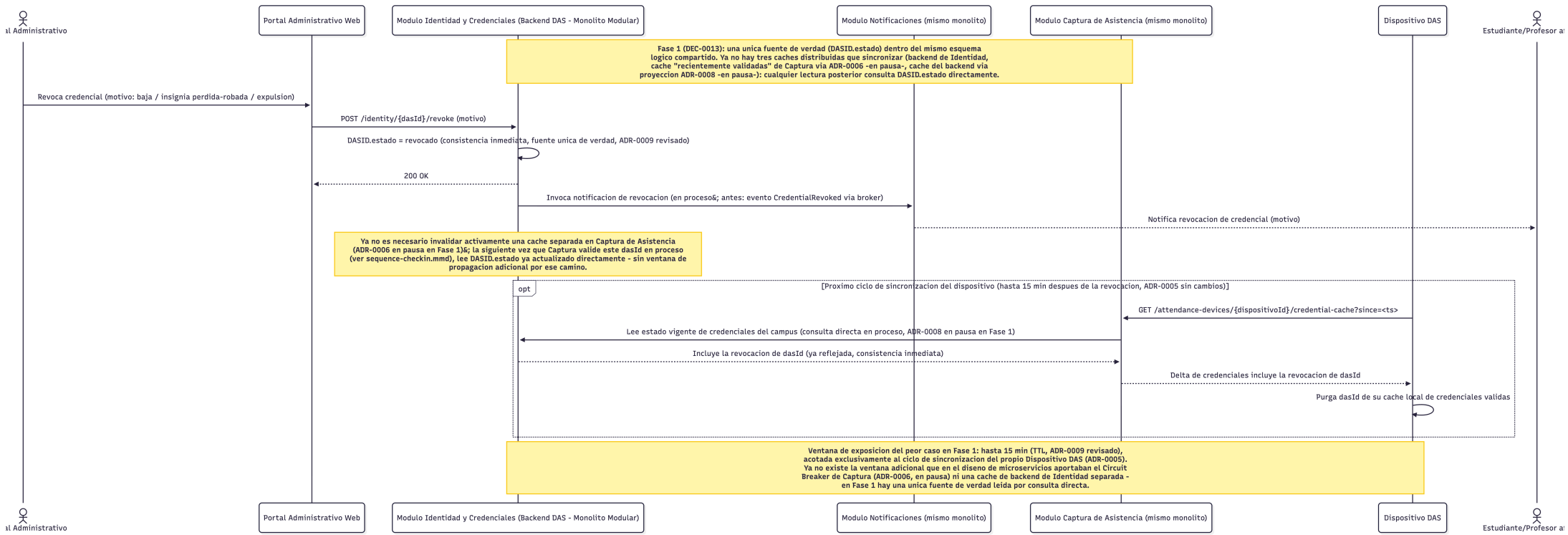
¿Por qué importa?

- Campus pueden operar sin red
- La asistencia no se pierde
- Al reconectar, los lotes **no desplazan** los escaneos en vivo
- `timestampEscaneo` + `timestampRecibido` permiten auditoría del orden real

Bulkhead de reconexión masiva

Límites de concurrencia, tamaño de lote y prioridad para la ruta crítica de captura en vivo.

Secuencia: Revocación de credencial



Ver diagrama completo

Datos y consistencia

Ownership de datos

Dato	Dueño
Solicitud de Registro	Registro y Verificación
DAS ID / Credencial	Identidad y Credenciales
Evento de Asistencia	Captura de Asistencia
Resúmenes / estadísticas	Reportes y Analítica
Registro de Notificación	Notificaciones

Consistencia por caso

Fuerte

Identidad, credencial, revocación y evento de asistencia: deben ser visibles de inmediato dentro del monolito.

Eventual

Reportes, estadísticas, cachés de dispositivos y resultado NIS: una demora acotada es preferible a detener la captura.

Ningún módulo escribe directamente datos de otro. Esa disciplina hace posible la migración futura.

Observabilidad

La observabilidad permite **operar DAS** y **decidir si debe evolucionar**. Las métricas son el criterio de extracción a Fase 2.

Operación

Latencia p95
Error rate
Rechazos de auth

Registro / NIS

Solicitudes pendientes
Timeouts y reintentos
Estado Circuit Breaker

Captura

Eventos recibidos
Duplicados detectados
Offline pendientes · Batch size

Reportes

Lag de proyección
Antigüedad de vistas

Negocio

Registros completados
Campus sin actividad
Asistencia procesada

Alertas críticas

NIS muy antiguo
Circuit Breaker abierto
Buffer cerca del límite

Riesgos y mitigaciones

Riesgo	Mitigación	Riesgo residual
Complejidad para equipo pequeño	Monolito modular hasta tener evidencia y madurez operativa	Extracción futura exige capacidad real
NIS incierto o indisponible	ACL · Jobs persistidos · Timeout · Retry · Escalamiento manual	Intervención humana depende del tercero
Buffer local lleno	Límites · Alertas · Estado degradado · Asistencia manual	Requiere procedimiento operativo de campus
Contención Captura / Reportes	CQRS interno · Lag medido · Posible extracción de Captura	Fase 1 sigue compartiendo proceso
Proceso / BD como punto único de falla	Alta disponibilidad · Backup/restauración · Monitoreo	Mayor radio de falla que servicios aislados

Trade-offs arquitectónicos

Decisión	Ganancia	Sacrificio aceptado
Monolito modular vs. microservicios	Menor costo operativo; límites claros	Escala como unidad; aceptable hasta que la carga pruebe lo contrario
Síncrono para captura, asíncrono para NIS/ reportes	Respuesta inmediata donde importa	Estados, jobs y monitoreo adicionales
Consistencia fuerte para identidad; eventual para vistas	Corrección en ops críticas; disponibilidad en captura	Reportes y cachés pueden tener retraso acotado
Buffer offline	Preservar asistencia durante cortes	Control de capacidad, orden y reconciliación
Extracción selectiva basada en evidencia	Pagar complejidad distribuida solo cuando aporta valor	Disciplina de contratos, métricas y migración gradual

¿Cómo evoluciona DAS?

La secuencia de evolución

Monolito Modular en operación
↓
Observabilidad activa
↓
Evidencia de presión real
↓
Identificación del módulo afectado
↓
Estabilización del contrato
↓
Extracción selectiva del módulo
↓
Migración de datos
↓
Comunicación distribuida donde se necesita
↓
Observar y decidir por módulo

Criterios para extraer Captura

Al menos UNO de estos:

- Más de 10 campus activos con carga próxima al escenario de prueba
- Degradación atribuible a contención Captura/ Reportes en 2 ventanas pico
- Necesidad repetida de desplegar Captura sin los demás módulos
- Pipeline e infraestructura reproducibles probados en producción

No se construye complejidad futura antes de necesitarla.

Roadmap

Hito	Resultado verificable	Riesgo que reduce
Validación de la propuesta	Drivers, restricciones, supuestos y escenarios priorizados	Diseñar sobre información incorrecta
Diseño de la solución	Arquitectura, bounded contexts, datos, integraciones y decisiones tecnológicas definidos	Contradicciones y decisiones sin justificar
Spike NIS	Contrato, entrega de resultado, errores y límites confirmados	Integración externa incorrecta
Implementación Fase 1	Monolito Modular con límites, ownership y flujos implementados	Acoplamiento y responsabilidades ambiguas
Validación de captura	Offline, idempotencia, orden y reconexión masiva probados	Duplicación o saturación de asistencia
Prueba de carga y seguridad	Meta near real-time, recuperación y controles medidos	Capacidad y privacidad no comprobadas
Operación Fase 1	Métricas, alertas y criterios de extracción activos	Migrar por intuición y no por evidencia

Fase 2 no tiene fechas fijas. Empieza únicamente cuando la operación y las métricas proporcionen evidencia suficiente para justificar la extracción selectiva de un bounded context.

Reflexión crítica sobre el uso de IA

Cuatro prompts principales

Prompt	Propósito
1 — Descubrimiento	Drivers, requisitos, ambigüedades
2 — Diseño †	Alternativas, decisiones, diagramas
3 — Trade-offs	Riesgos, costos, mitigaciones
4 — Síntesis	Defensa, narrativa, informe

† Mayor valor: Prompt 2, por separar proposición, modelado y revisión adversarial.

Lo que requirió control manual

- Supuestos que se convertían en requisitos
- Patrones propuestos sin driver que los justifique
- Riesgos con soluciones que aumentaban complejidad
- Respuestas plausibles sin evidencia suficiente

Aprendizaje principal

La IA generó mayor valor como **herramienta de razonamiento y revisión**, no como generador de contenido. Una respuesta más completa no es necesariamente una respuesta mejor.

Muchas gracias